KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG - UIT

BÁO CÁO THỰC HÀNH

Lab 4

CÔNG NGHỆ INTERNET OF THINGS HIỆN ĐẠI

Xây dựng giao diện ứng dụng để tương tác với mô hình IoT

GVHD: **Phan Trung Phát**

Lớp: **NT532.O21.MMCL.2**

Họ và tên	MSSV
Tống Võ Anh Thuận	21522652
Trịnh Vinh Đại	21521915
Lê Huỳnh Quanh Vũ	21522797

ĐÁNH GIÁ KHÁC (*):

Nội dung	Kết quả
Tổng thời gian thực hiện bài thực hành trung bình (1)	
Link Video thực hiện (2) *(nếu có)*	Code_n_Video
Ý kiến (3) *(nếu có)* + Khó khăn + Đề xuất …	
Điểm tự đánh giá (4)	10/10

(*): phần (1) và (4) bắt buộc thực hiện.

Phần bên dưới là báo cáo chi tiết của nhóm/cá nhân thực hiện.

WARNING LƯU HÀNH NỘI BỘ

© Phan Trung Phát - phatpt@uit.edu.vn

Lựa chọn 1 nền tảng IoT platform mã nguồn mở và thực hiện xây dựng ứng dụng IoT cơ bản đầy đủ các thành phần và chức năng, bao gồm:

The IoT platform we will be using for this lab is **ThingsBoard Cloud**. It is free and open source (it is also recommended by the instructor). Documentation is available here: https://thingsboard.io/docs/paas/

1. Device: sinh viên thực hiện sử dụng 2 loại thiết bị cảm biến DHT22/DHT11 và MQ-135 để lấy 3 loại dữ liệu cảm biến là nhiệt độ, độ ẩm và ánh sáng. Trên Wemos D1 sử dụng cảm biến BH-1750, đồng thời lập trình để gửi các giá trị cảm biến đến hệ thống thông qua giao thức HTTP. Trên Raspberry Pi sử dụng DHT22/DHT11, đồng thời lập trình để gửi các giá trị cảm biến đến hệ thống thông qua giao thức MQTT. Các dữ liệu này cũng được lưu trữ tại hệ thống IoT platform.

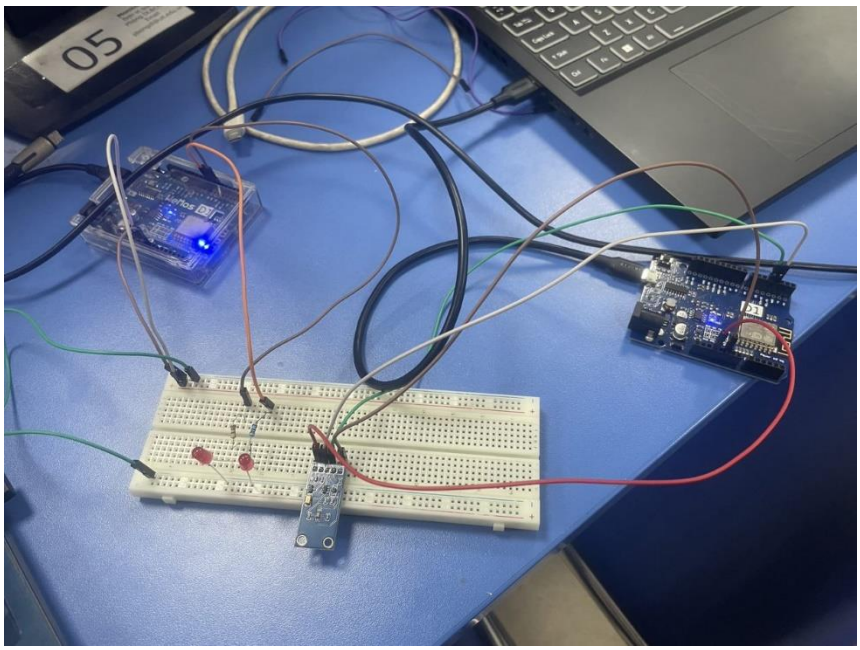
a) Wemos D1

Set up:
Components in used:

- 2 Wemos D1 R2.
- 1 BH1750 sensor.
- 2 LEDs.
- 2 resistors.
- 6 jumper wires

Wire connection:

- 2 LEDs:
  - Anode: Use digital pins D3 and D4 on Wemos 2. Those pins control LED output. Each LED connects with one resistor.
  - Cathode: Connect to GND zone on the breadboard. This zone grounds the LED.
- BH1750 sensor:
  - Use digital pins SCL/D1 and SDA/D2 on Wemos 1.
  - Anode and Cathode: Connect to GND zone and VCC zone on the breadboard.



Code explanation:

Wemos 1: connect with BH1750 sensor.

```
const char* ssid = "Daivjppro";
const char* password = "123456780";
const char* mqtt_server = "mqtt.thingsboard.cloud";
const char* topic = "v1/devices/me/telemetry";
const String username = "1n2DNxT7sDDA0ycBziSe";

```

This code declares Wifi name, password, MQTT server address, topic and username.

```
void setup() {
  Serial.begin(115200);
  Wire.begin(D2, D1); // Khởi động giao tiếp I2C với chân SDA là D2 và chân SCL là D1
  if (!lightMeter.configure(BH1750::CONTINUOUS_HIGH_RES_MODE)) {
    Serial.println("BH1750 configuration failed!");
  } else {
    Serial.println("BH1750 configured successfully!");
  }
  setup_wifi(); // Kết nối WiFi
  client.setServer(mqtt_server, 1883); // Thiết lập máy chủ MQTT
}

void loop() {
  if (!client.connected()) {
    reconnect(); // Kết nối lại nếu mất kết nối MQTT
  }
  client.loop();

  // Đọc giá trị từ cảm biến BH1750
  float lux = lightMeter.readLightLevel();

  // Gửi giá trị lên MQTT
  char msg[50];
  snprintf (msg, 50, "{\"lux\":%.2f}", lux);
  Serial.print("Publish message: ");
  Serial.println(msg);
  client.publish(topic, msg);

  delay(2000); // Chờ 2 giây trước khi gửi lại
}
```

This code snippet is to start I2C communication, set up the MQTT Server, try to connect to the MQTT Server again if the connection fails, read the BH1750 sensor value, and send data to the Server.

Wemos 2: connect with 2 LEDs.

```
#define WIFI_AP "Daivjppro"
#define WIFI_PASSWORD "123456780"

#define TOKEN "1n2DNxT7sDDA0ycBziSe"

#define GPIO0 0
#define GPIO2 2

#define GPIO0_PIN 3
#define GPIO2_PIN 5

char thingsboardServer[] = "mqtt.thingsboard.cloud";
const char* topic = "v1/devices/me/rpc/request/+";  //
```

This code declares Wifi name, password, MQTT server address, topic, define GPIO pins and define pin codes for those 2 GPIO pins.

```
void on_message(char* topic, byte* payload, unsigned int length) {

  Serial.println("On message");

  char json[length + 1];
  strncpy (json, (char*)payload, length);
  json[length] = '\0';

  Serial.print("Topic: ");
  Serial.println(topic);
  Serial.print("Message: ");
  Serial.println(json);

  // Decode JSON request
  DynamicJsonDocument jsonBuffer(200); // Sử dụng DynamicJsonDocument
  DeserializationError error = deserializeJson(jsonBuffer, json);

  if (error)
  {
    Serial.println("deserializeJson() failed");
    return;
  }

  // Check request method
  String methodName = jsonBuffer["method"].as<String>();

  if (methodName.equals("getGpioStatus")) {
    // Reply with GPIO status
    String responseTopic = String(topic);
    responseTopic.replace("request", "response");
    client.publish(responseTopic.c_str(), get_gpio_status().c_str());
  } else if (methodName.equals("setGpioStatus")) {
    // Update GPIO status and reply
    set_gpio_status(jsonBuffer["params"]["pin"].as<int>(), jsonBuffer["params"]["enabled"].as<boolean>());
    String responseTopic = String(topic);
    responseTopic.replace("request", "response");
    client.publish(responseTopic.c_str(), get_gpio_status().c_str());
    client.publish("v1/devices/me/attributes", get_gpio_status().c_str());
  }
}
```

This code will do the following:

- Create a function to receive messages from MQTT Server.
- Handle the payload: Create a char array of length equal to payload + 1 to contain the JSON message, then copy the payload content into the JSON array and add the string terminator character "\0" to make it a valid string.
- Print to Serial port the information received.
- JSON decoding: Use DynamicJsonDocument to create a 200-byte dynamic JSON document, then decode the JSON string and check for decoding errors.
- Handle request method:
- Reply to request:

```
String get_gpio_status() {
  // Prepare gpios JSON payload string
  DynamicJsonDocument jsonBuffer(200); // Sử dụng DynamicJsonDocument
  JsonObject data = jsonBuffer.to<JsonObject>();
  data[String(GPIO0_PIN)] = gpioState[0] ? true : false;
  data[String(GPIO2_PIN)] = gpioState[1] ? true : false;
  String strPayload;
  serializeJson(data, strPayload);
  Serial.print("Get gpio status: ");
  Serial.println(strPayload);
  return strPayload;
}

void set_gpio_status(int pin, boolean enabled) {
  if (pin == GPIO0_PIN) {
    // Output GPIOs state
    digitalWrite(GPIO0, enabled ? HIGH : LOW);
    // Update GPIOs state
    gpioState[0] = enabled;
  } else if (pin == GPIO2_PIN) {
    // Output GPIOs state
    digitalWrite(GPIO2, enabled ? HIGH : LOW);
    // Update GPIOs state
    gpioState[1] = enabled;
  }
}

void InitWiFi() {
  Serial.println("Connecting to AP ...");
  // attempt to connect to WiFi network

  WiFi.begin(WIFI_AP, WIFI_PASSWORD);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("Connected to AP");
}
```

- Creates and returns a JSON string containing the current state of the GPIO pins.
- Set the state for a specific GPIO pin and update this state in the "gpioState" array.
- Connect the device to the WiFi network.

b) Jetson Nano

Reason for the alternative choice:

1. Both Jetson Nano and Raspberry Pi are both single-board computers (SBCs), so the purpose of use remains unchanged.

2. They run the same OS distro, which is Debian, so the development environment should be the same.
3. Jetson Nano has a better performance since it has 4 GB 64-bit LPDDR4 with a strong CPU.
4. We own Jetson Nano locally but not Raspberry Pi so we are flexible to complete the LAB on time.

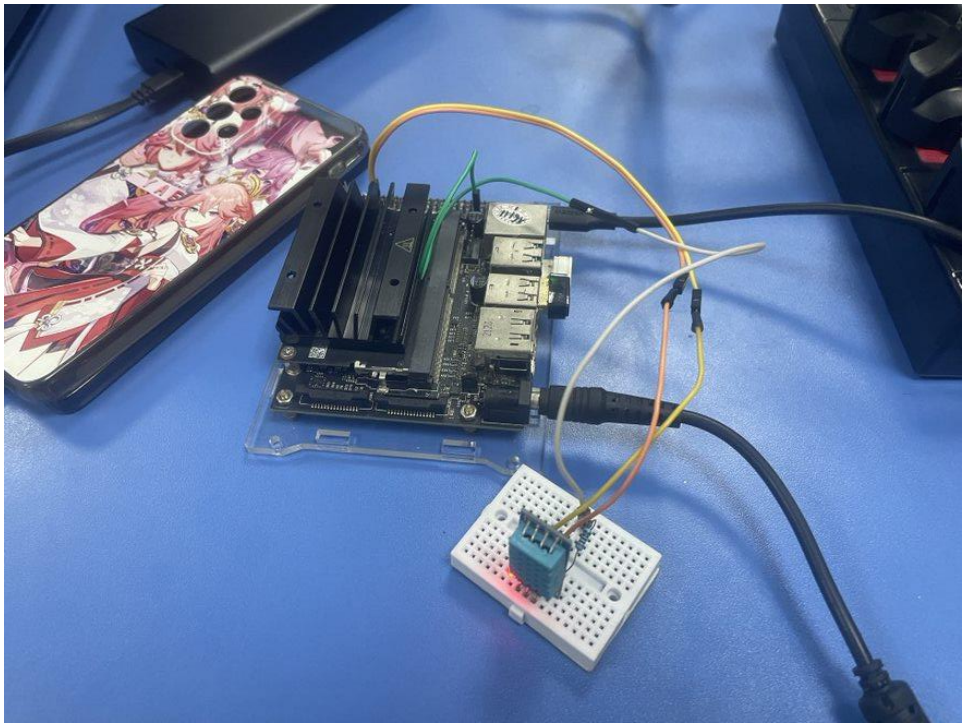
Set up:

Components in use:

- 1 Jetson Nano B1.
- 1 DHT11 sensor.
- 1 resistor.
- 6 jumper wires

Wire connection:

- DHT11 sensor:
    - GND: connect to GND pin on Jetson Nano. This helps ground sensor.
    - Data: connect to pin 33, which is GPIO13 on Jetson Nano. This help transfer data to Jetson Nano.
    - VCC: connect to 5V pin through a resistor on Jetson Nano. This helps power sensor.



Code explanation:

```
import C_CHT
import time
from tb_device_mqtt import TBDeviceMqttClient, TBPublishInfo

while True:

    data = C_DHT.readSensorDHT11(0);
```

```
    # data = {temperature, NULL, humidity, NULL}
    telemetry = {"temperature": data[0], "humidity": data[2]}
    client = TBDeviceMqttClient("THINGBOARD-SERVER", "THINGBOARD-TOKEN")
    # Connect to ThingsBoard
    client.connect()
    # Sending telemetry without checking the delivery status
    client.send_telemetry(telemetry)
    # Sending telemetry and checking the delivery status (QoS = 1 by
default)
    result = client.send_telemetry(telemetry)
    # get is a blocking call that awaits delivery status
    success = result.get() = TBPublishInfo.TB_ERR_SUCCESS
    # Disconnect from ThingsBoard
    client.disconnect()
    time.sleep(1)
```

Library in use:

1. C_CHT: This is a C-to-Python library, which is built specifically for Jetson Nano to read DHT11/22 sensor.
2. TBDeviceMqttClient, TBPublishInfo: Those are Thingboards Python libraries that are used to connect and publish data to the Thingboards Server through MQTT.
3. Time: Normal Python library, to delay loop function.

Main code: This code acts as a cron job, which executes every 1 second.

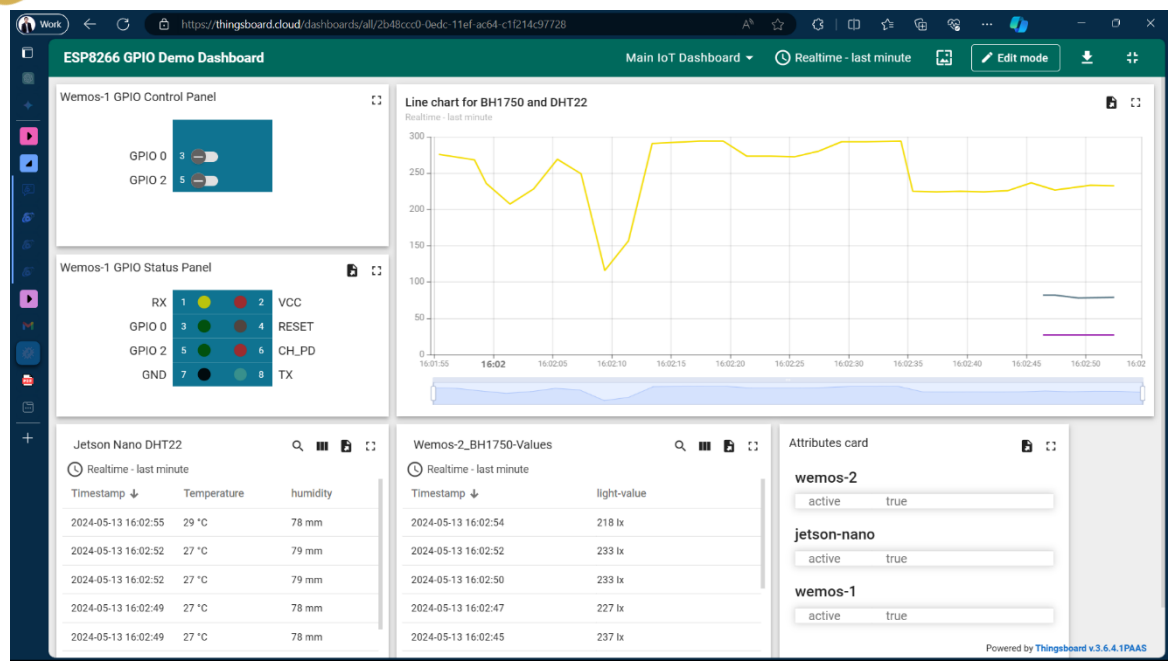
- First, Jetson Nano read data from DHT11 in first option (0). There are 2 pin (33, 18) on Jetson Nano that can read DHT sensor, so that option (0) and option (1) respectively.
- Second, extract data and assign it to telemetry for sending to the server.
- Third, connect Jetson Nano to ThingBoards Server by providing the server address and authentication token.
- Fourth, send telemetry to the server and get response from the server.
- Finally, release the connection and repeat the process.

2. Dashboard: hiển thị tình trạng thiết bị đang kết nối đến hệ thống (Wemos D1, Raspberry Pi,...).

Lưu ý: số lượng thiết bị kết nối đến hệ thống phải từ 2 trở lên (2 Wemos D1,...).

As you can see in the dashboard below, we have “Attribute card” (bottom right) which shows the connection status of 3 currently active IoT devices . Besides, we also have:

- GPIO Control and Status Panel to control the LEDs of the ESP8266 device.
- A line chart to visualize data from the 2 sensors BH1750 and DHT11
- Two time-series tables for the 2 sensors as well (Humidity is incorrectly set as mm for unit due to a mistake).



All the explanations after this section will use the above dashboard image as a reference.

3. Màn hình chính: dùng để thể hiện các tương tác với hệ thống, bao gồm: thể hiện các giá trị cảm biến ở thời điểm hiện tại (tên giá trị, giá trị của cảm biến), chức năng điều khiển 2 đèn LED trên Wemos D1 thông qua giao diện ứng dụng. Chức năng này phải thể hiện thời gian thực.

For LED controlling, both the dashboard configuration and the Wemos code is referenced here: https://thingsboard.io/docs/samples/esp8266/gpio/

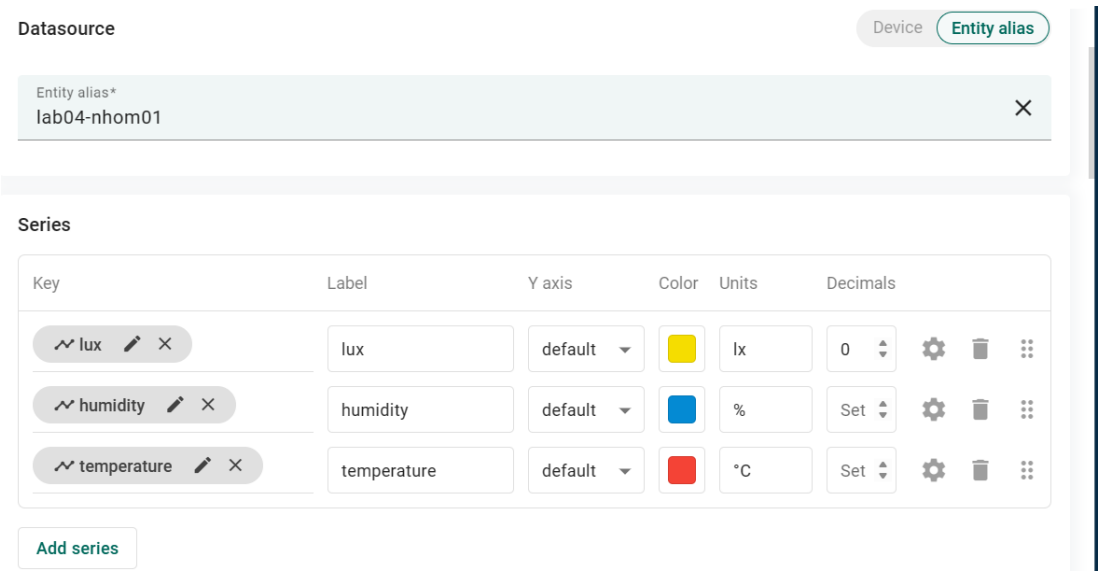
There are two widgets used for LED controlling in the dashboard:

- ESP8266 GPIO Control Panel (row 0, col 0):
  - This widget allows you to control individual GPIO pins (3 and 5) on the ESP8266 device.
  - It uses an RPC (Remote Procedure Call) type for communication.
  - Clicking a switch on the widget sends a request to the device to change the pin state (on/off).
- ESP8266 GPIO Status Panel (row 0, col 5):
  - This widget displays the current status (on/off) of various GPIO pins on the ESP8266 device.
  - It uses a "latest" type, indicating it shows the most recent data received.
  - Each pin has a label (e.g., GPIO 0, VCC) and a configurable color for better visualization.

4. Biểu đồ: dùng để thể hiện các giá trị cảm biến thu thập được từ phần thiết bị một cách trực quan bằng biểu đồ, bảng biểu. Cụ thể như sau: các giá trị cảm biến bao gồm 3 loại (nhiệt độ, độ ẩm, ánh sáng). Các biểu đồ này thể hiện dưới dạng thời gian thực.

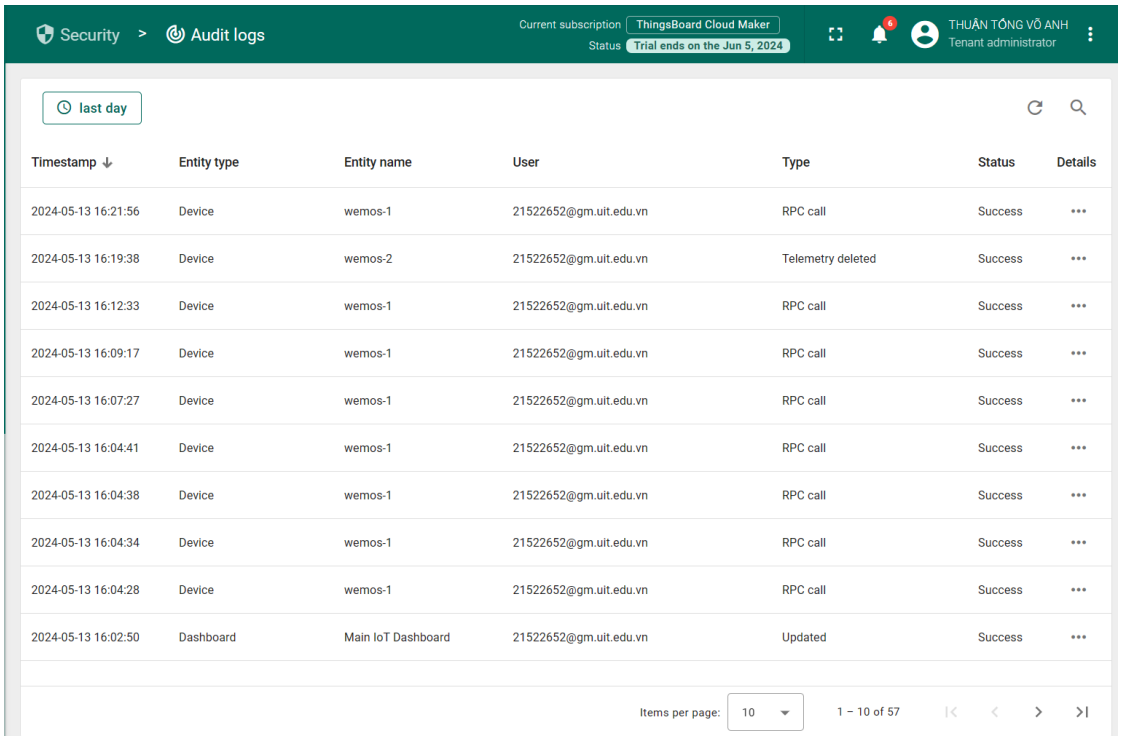
In order to draw the line chart like in the dashboard image. We simply create a line chart widget with the datasource being “lab04-nhom01” which includes all 3

IoT devices (2 Wemos, 1 Jetson Nano). Then we set the key, label, color, and unit for the sensor values for lux, humidity, and temperature respectively.

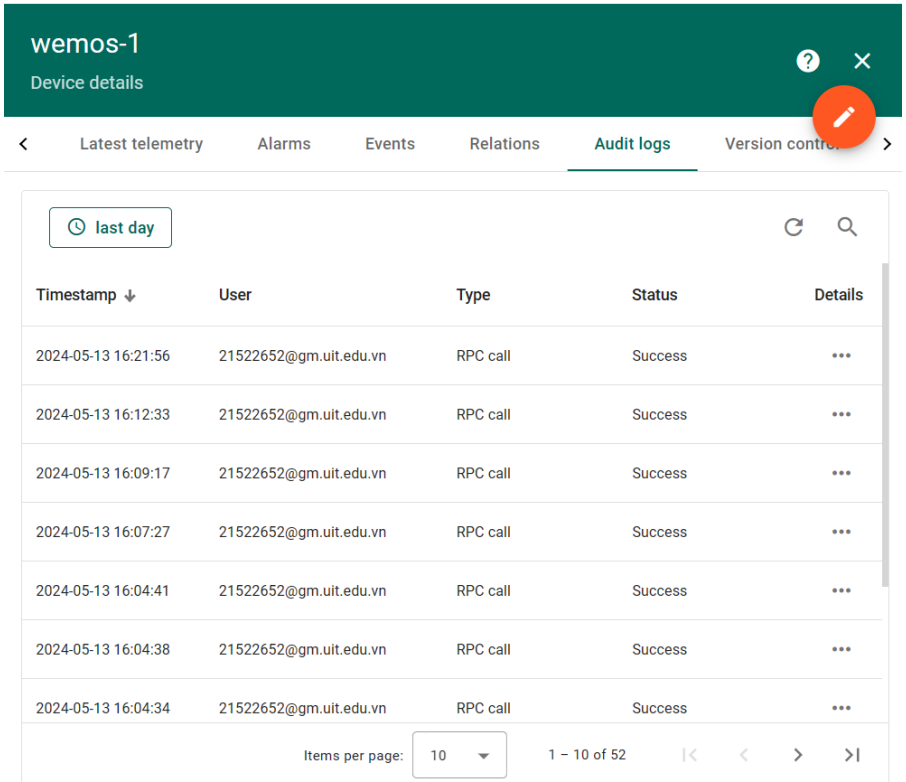


5. Logs: thể hiện các log của hệ thống để người quản trị có thể dễ dàng tương tác, quản lý đối với hệ thống. Cụ thể như sau: ghi lại các thông tin của hệ thống, giá trị cảm biến từ phần thiết bị gửi đến, các trường dữ liệu cần phải có: ID của thiết bị, tên thiết bị, tên giá trị, giá trị cảm biến, thời gian nhận dữ liệu.

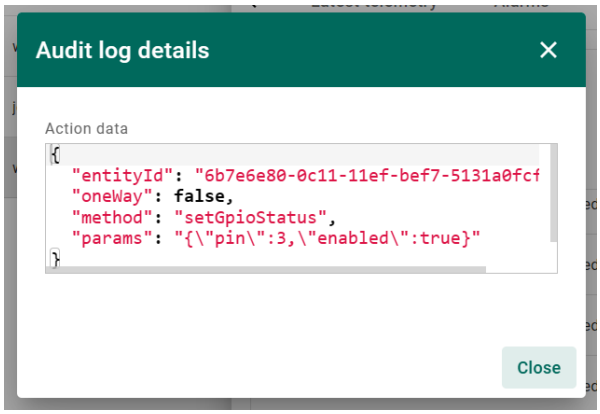
To see logs of changes made to the system, on the web UI, go to the Security section, choose Audit logs. The result will be similar to the image below.



You can also go to the Devices page to see the audit logs of each device:



To see details of each log entry, click on the "..." button on the far right and the result will be shown in JSON format, including the ID of the entity you are viewing along with some attributes:



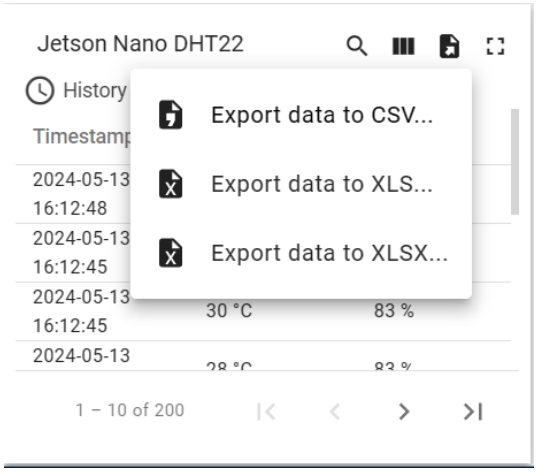
For logs related to sensor values, this type of data is available in the widget data which is the same content for Question 7 (for both viewing and downloading) so it will not be mentioned here.

6. (Nâng cao) thực hiện triển khai IoT platform đó ở local.

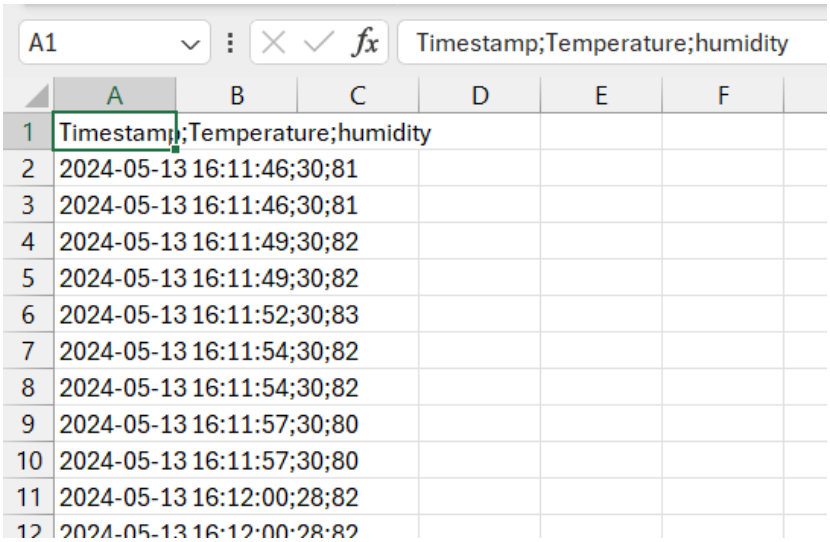
We couldn't do this due to skill issue so there's that (the platform on local doesn't have the download function built-in for us).

7. (Nâng cao) Người dùng có thể lọc lại các dữ liệu mà họ mong muốn hoặc download toàn bộ dữ liệu dưới các định dạng như csv, excel,...

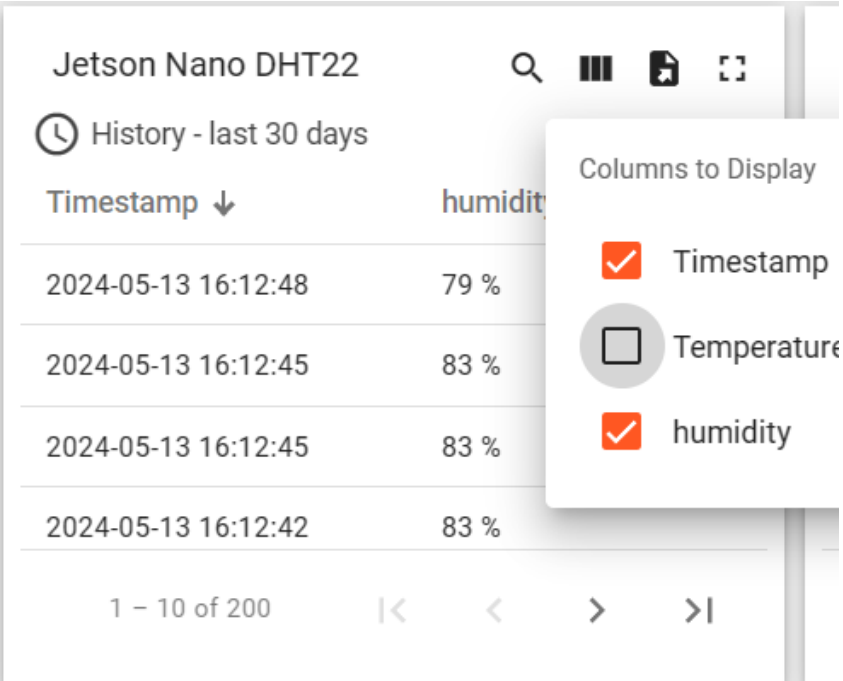
To do this, go to the dashboard and choose the widget data that you want to save. Click on the download icon on the top right corner of the widget and you will see some download options like below:



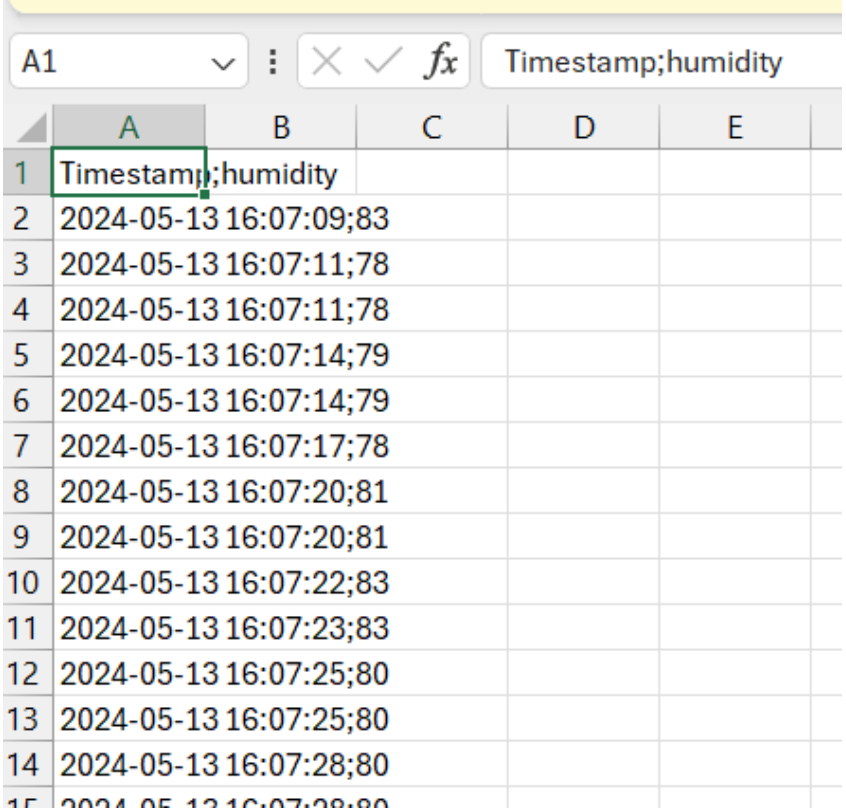
The csv file once downloaded:



If you want to filter data of a certain type, you can do so with the column display option. In the image below, I remove temperature from the display and thus, after downloading, only humidity data shall be included.



The content of the downloaded csv file only contains data of humidity as shown below:



YÊU CẦU CHUNG

1) Đánh giá

- Chuẩn bị tốt các yêu cầu đặt ra trong bài thực hành.
- Sinh viên hiểu và tự thực hiện được bài thực hành, trả lời đầy đủ các yêu cầu đặt ra.
- Nộp báo cáo kết quả chi tiết những đã thực hiện, quan sát thấy và kèm ảnh chụp màn hình kết quả *(nếu có)*; giải thích cho quan sát *(nếu có)*.
- Sinh viên báo cáo kết quả thực hiện và nộp bài.

2) Báo cáo

- File .PDF hoặc .docx. Tập trung vào nội dung, giải thích.
- Nội dung trình bày bằng Font chữ Cambria hoặc Times New Roman (*tuy nhiên, phải chuyển đổi hết báo cáo này sang 1 font chữ thống nhất*) – cỡ chữ 13. Canh đều (Justify) cho văn bản. Canh giữa (Center) cho ảnh chụp.
- Đặt tên theo định dạng: Mã lớp-LabX_MSSV1_MSSV2. (trong đó X là Thứ tự buổi Thực hành).

  Ví dụ: NT532.O21.1-Lab01_25520001_25520002
- Nếu báo cáo có nhiều file, nén tất cả file vào file .ZIP với cùng tên file báo cáo.
- Không đặt tên đúng định dạng – yêu cầu, sẽ KHÔNG chấm điểm bài thực hành.
- Nộp file báo cáo trên theo thời gian đã thống nhất tại courses.uit.edu.vn.

Bài sao chép, trễ, … sẽ được xử lý tùy mức độ vi phạm.

HẾT